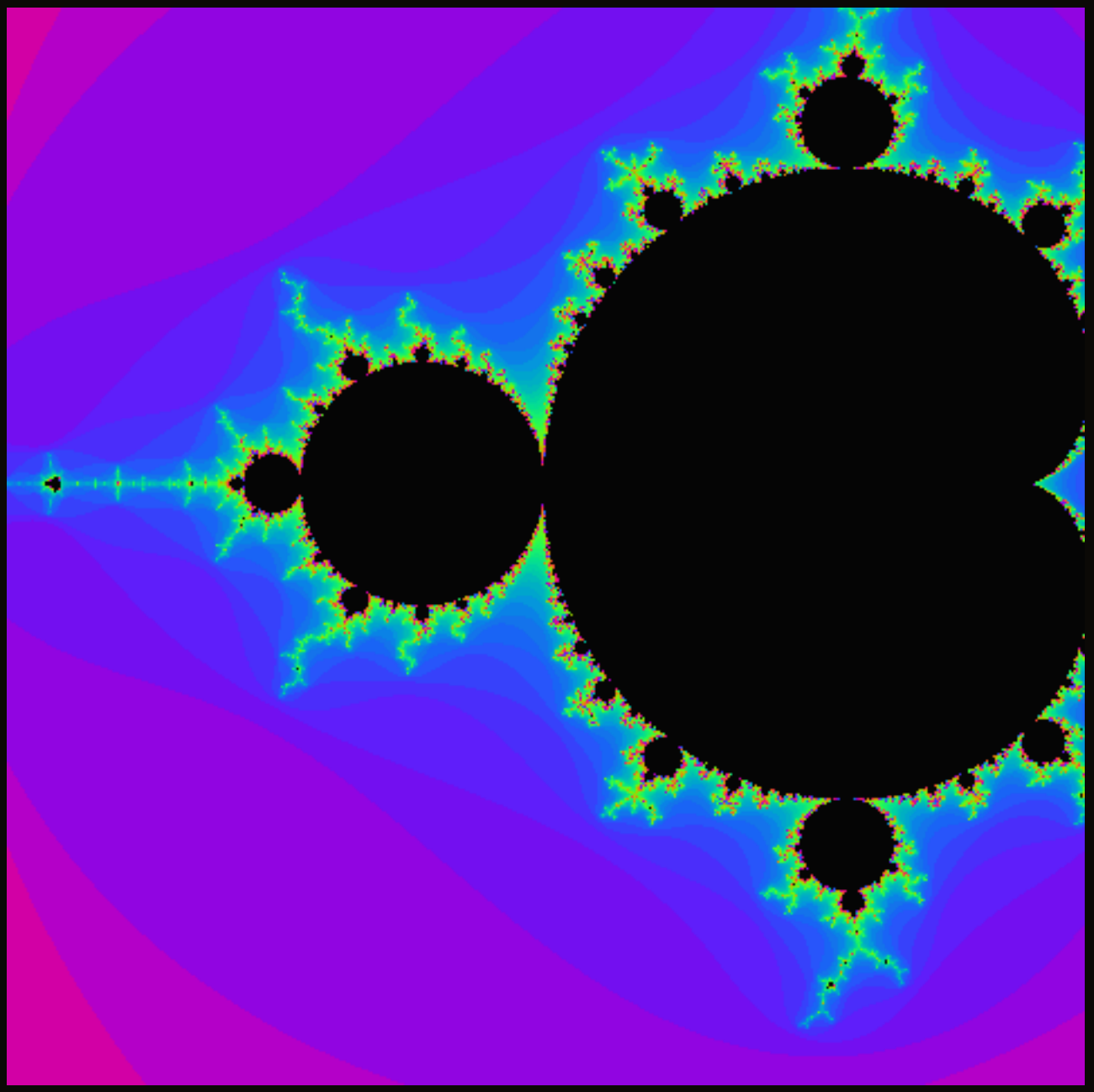
NO GPU · NO GRAPHICS LIBRARY · NO OS

We drew this on a computer with no operating system.

One program, 16 MiB of RAM, and a socket. Nothing else is running — there is nothing else to run.

The PNG is encoded by hand, because there was no image library to call.

FRAME 1, STRAIGHT OFF THE MACHINE



480x480 · palette PNG written byte by byte in C

this copy drawn by the Linux build of the same source — see slide 12

WHAT IS ACTUALLY UNDERNEATH

A unikernel. The program is the machine.

No kernel. No shell. No init, no container, no package manager, no language runtime. The hypervisor loads a 2.8 MB image and execution begins in main().

So there is no libpng to link, no zlib, no framebuffer device. If you want a picture, you compute every pixel and you write the file format yourself.

THE PNG ENCODER, IN FULL

Deflate has a mode that does nothing.

Porting zlib to call one function is the larger job. But a deflate stored block is a five-byte header wrapped around at most 65,535 literal bytes — no compression, no Huffman tree, no tables.

A zlib stream is a two-byte header, a run of those, and an Adler-32. Every decoder on earth accepts it.

CRC32 table, 20 lines. Encoder, about 80. Validated against a real decoder, not just a viewer that opened it.

THE PART THAT IS NOT DECORATION

Every frame is rendered twice.

An escape count is a pure function of the pixel. Render the same frame twice and a correct machine returns the same image, byte for byte, *by construction*.

We had already measured this platform disagreeing with itself on fixed arithmetic — about one result in four thousand, with a Linux control under the same hypervisor coming back clean.

So: render twice, compare, paint every disagreeing pixel red. The picture is the measurement.

AND THE ANSWER WAS

0

red pixels. Every frame, every build.

Which is a result, not a disappointment. The corruption we measured is in 64-bit integer modular arithmetic — gcc lowers it to a __udivti3 call.

A Mandelbrot loop is double-precision floating point, in SSE registers. Different path. So far, an uncorrupted one.

THEN IT WOULD NOT SEND

787 KB left the machine and never arrived.

```
[!] send: Failure when receiving data from the peer
[!] frame 1 not delivered
```

The same binary, the same frame, posted from Linux on the same host: fine. So it was the unikernel's network stack, not the API at the other end.

WRONG CONCLUSION #1

"It's too big. I need a real compressor."

Confidently reasoned from a single failure. A fixed-Huffman deflate encoder, five to ten times smaller, about a day of work.

I did not write it. I measured the wall instead — which is the only good decision in this entire sequence.

WRONG CONCLUSION #2

A ladder said 256 KB. To the wrong server.

```
PROBE 262144 bytes  ok   HTTP 401
[+] largest body: 256 KB
```

Measured against our own service, then generalised to a machine-wide ceiling. The next frame was 231 KB, comfortably under it, and failed anyway.

WRONG CONCLUSION #3

A second ladder said 16 KB.

```
PROBE  12288 bytes  ok      HTTP 400
PROBE  16384 bytes  FAILED  (31s)
```

Pointed at Telegram this time. Better. Then a 5,008-byte frame failed too — and no size hypothesis survives that.

Size was never the variable. It had just been big enough, every time, to look like one.

THE ACTUAL BUG

9

bytes into every PNG, there is a zero.

The probe bodies were ASCII. The frames are binary. CURLOPT_POSTFIELDS in this port stops at the first NUL — while Content-Length still promises the rest, so the server waits for data that never comes, then closes.

Every frame failed at every size. That is what "no size hypothesis survives" was telling me.

SIX LINES, AND IT FLEW

```
struct upload up = { g_body, g_body_len };
curl_easy_setopt(h, CURLOPT_READFUNCTION, read_cb);
curl_easy_setopt(h, CURLOPT_READDATA, &up);
```

Feed the body a chunk at a time instead of handing over a pointer and a length. It is also what curl's own documentation recommends for anything that is not a C string.

AND NOW THE SIZE LIMIT IS REAL

```
[+] ladder  64x64  png=  5008  posted
[+] ladder  96x96  png= 10160  posted
[+] ladder 128x128 png= 17360  FAILED
[+] settled on 96x96
```

Two bugs, not one. The binary bug was hiding a genuine ceiling near 16,384 bytes — which is exactly the TLS maximum record size.

SO IT SIZES ITSELF

The program finds its own limit, by posting.

It renders at 64, posts it, steps up, and keeps stepping until one fails. The largest that survives becomes the size it runs at.

Not a number I hardcoded from a measurement that might go stale — a number the machine establishes about itself, every time it boots.

Today that is 96x96. If the port is fixed, it will find that out on its own.

WHAT THIS COST, AND WHAT IT BOUGHT

Three wrong answers. Two real bugs.

Every wrong conclusion came from real data, generalised one step too far. What broke the sequence was not a better guess — it was a 5 KB failure small enough that no size story could explain it.

Measure the thing that is failing. Not the thing that is convenient to measure.

IT IS RUNNING NOW

Zooming 1.35x a frame, into the seahorse valley.

> github.com/tabibazar/baremetal-agent

MIT. The renderer, the hand-written PNG encoder, the probe, the deploy scripts.

Built on Return Infinity's BareMetal. All three bugs reported, each with a reproducer.